

Funciones en MATLAB y ejemplos prácticos

1. `int`

- **Descripción:** Calcula integrales indefinidas y definidas.
 - `int(f, x)` : Calcula la integral indefinida de f con respecto a x .
 - `int(f, x, a, b)` : Calcula la integral definida de f entre los límites a y b .
- **Ejemplo:**

matlab

 Copiar código

```
syms x
int(x*exp(x))      % Integral indefinida
int(x*exp(x), 0, 1) % Integral definida de 0 a 1
```

2. `solve`

- **Descripción:** Resuelve ecuaciones simbólicas.
- **Ejemplo:**

matlab

 Copiar código

```
syms x
solve(x^2 - 4) % Soluciones de x^2 - 4 = 0
```

3. `fplot`

- **Descripción:** Representa gráficamente funciones de una variable.
- **Ejemplo:**

matlab

 Copiar código

```
syms x
f(x) = sin(x);
fplot(f, [-pi, pi]) % Gráfica de sin(x) en el intervalo [-pi, pi]
```

4. `area`

- **Descripción:** Dibuja y colorea el área entre la gráfica de una función y el eje X.
- **Ejemplo:**

matlab

 Copiar código

```
syms x
f(x) = sin(x);
xd = linspace(-pi, pi, 50);
yd = double(f(xd));
area(xd, yd) % Dibuja el área sombreada
```

5. `taylor`

- **Descripción:** Calcula polinomios de Taylor o MacLaurin.
 - `'order', n`: Especifica el grado del polinomio.
- **Ejemplo:**

matlab

 Copiar código

```
syms x
f(x) = exp(-x^2);
p6 = taylor(f, 'order', 7) % Polinomio de MacLaurin de grado 6
```

7. fsurf

- **Descripción:** Representa superficies en 3D.
 - 'MeshDensity', n : Ajusta la densidad de la malla.
 - 'ShowContours', 'on' : Añade curvas de nivel.
- **Ejemplo:**

matlab

 Copiar código

```
syms x y
f(x, y) = x^2 + y^2;
fsurf(f, [-2, 2, -3, 3], 'ShowContours', 'on') % Superficie con curvas de nivel
```

8. gradient

- **Descripción:** Calcula el gradiente de una función, proporcionando un vector columna.
- **Ejemplo:**

matlab

 Copiar código

```
syms x y
f(x, y) = x^2 + x*y^2;
grad = gradient(f) % Vector gradiente de la función
```

9. diff

- **Descripción:** Calcula derivadas parciales o de mayor orden.
- **Ejemplo:**

matlab

 Copiar código

```
syms x y
f(x, y) = x^2 + sin(x*y);
dx = diff(f, x) % Derivada parcial respecto a x
```

10. `hessian`

- **Descripción:** Calcula la matriz Hessiana de una función.
- **Ejemplo:**

matlab

 Copiar código

```
syms x y
f(x, y) = x^2 + y^2 + x*y^2;
H = hessian(f) % Matriz Hessiana de la función
```

11. `symsum`

- **Descripción:** Calcula la suma de una serie simbólica.
- **Ejemplo:**

matlab

 Copiar código

```
syms n
f(n) = 1 / (n^2 + 3*n + 2);
symsum(f, n, 1, inf) % Suma infinita desde n=1
```

FUNCIONES BÁSICAS

Notación científica	Nombre en MATLAB	Significado
$ x $	<code>abs (x)</code>	valor absoluto de x
$\text{sen } x$	<code>sin (x)</code>	seno de x
$\text{cos } x$	<code>cos (x)</code>	coseno de x
$\text{tg } x$	<code>tan (x)</code>	tangente de x
$\text{arc sen } x$	<code>asin (x)</code>	arcoseno de x
$\text{arc cos } x$	<code>acos (x)</code>	arcocoseno de x
$\text{arc tg } x$	<code>atan (x)</code>	arcotangente de x
e^x	<code>exp (x)</code>	exponencial de x
$\text{Ln } x$	<code>log (x)</code>	logaritmo en base e de x
$\sqrt{x}$	<code>sqrt (x)</code>	raíz cuadrada de x

EXPRESIONES SIMBÓLICAS

<code>syms x y z</code>	Crea las variables simbólicas x , y , z .
<code>solve (Expr)</code>	Calcula los ceros de Expr.
<code>solve (Expr, z)</code>	Despeja z en la expresión Expr.
<code>subs (S, x, a)</code>	Sustituye en la expresión simbólica S la variable x por a .
<code>pretty (S)</code>	Presenta de forma <i>guapa</i> la expresión S .
<code>double (S)</code>	Calcula el valor numérico de la expresión simbólica S .
<code>simplify (S)</code>	Simplifica la expresión simbólica S .
<code>expand (S)</code>	Desarrolla la expresión S como producto de sus factores.
<code>f (x) = Expr</code>	Define la función simbólica f que depende de la variable simbólica x , entre muchas otras cosas, nos permitirá calcular el valor de f en a escribiendo $f (a)$.
<code>solve (f (x) == 0)</code>	Resuelve la ecuación $f (x) = 0$, siendo $f (x)$ una función simbólica.

CÓMO HACE PARA PASAR UNAS LÍNEAS DE GRÁFICO POR EL ORIGEN

Lo pongo porque, aunque sea una tontería hay que tenerlo en cuenta.

Tras tener nuestra función, indicamos con FPLOTT la representación gráfica de la función con un intervalo y un color. Después, ponemos estas tres líneas de código:

```
ax = gca;
```

```
ax.XAxisLocation = 'Origin';
```

```
ax.YAxisLocation = 'Origin';
```

DIFERENCIA ENTRE PLOT Y FPLOTT

<code>plot(x, y, S)</code>	Dibuja el conjunto de puntos (x_i, y_i) , donde x e y son vectores fila del mismo número de coordenadas, con las opciones especificadas en S . En S se describe entre comillas simples, el tipo de trazo, el color y el tipo de marca de los puntos usados.
<code>fplot(f, [xmin, xmax])</code>	Dibuja la función f , en el intervalo $[xmin, xmax]$.

Por ejemplo, para dibujar la función $y = x^2$ en el intervalo $[-3, 3]$

```
>> syms x
>> fplot(x^2, [-3, 3])
```

Para dibujar el punto (2,4) con un asterisco rojo escribimos

```
>> plot(2, 4, 'r*')
```

HOLD ON

No cierra la ventana gráfica para que las siguientes órdenes de representación gráfica se añadan sobre ella.

Axis equal

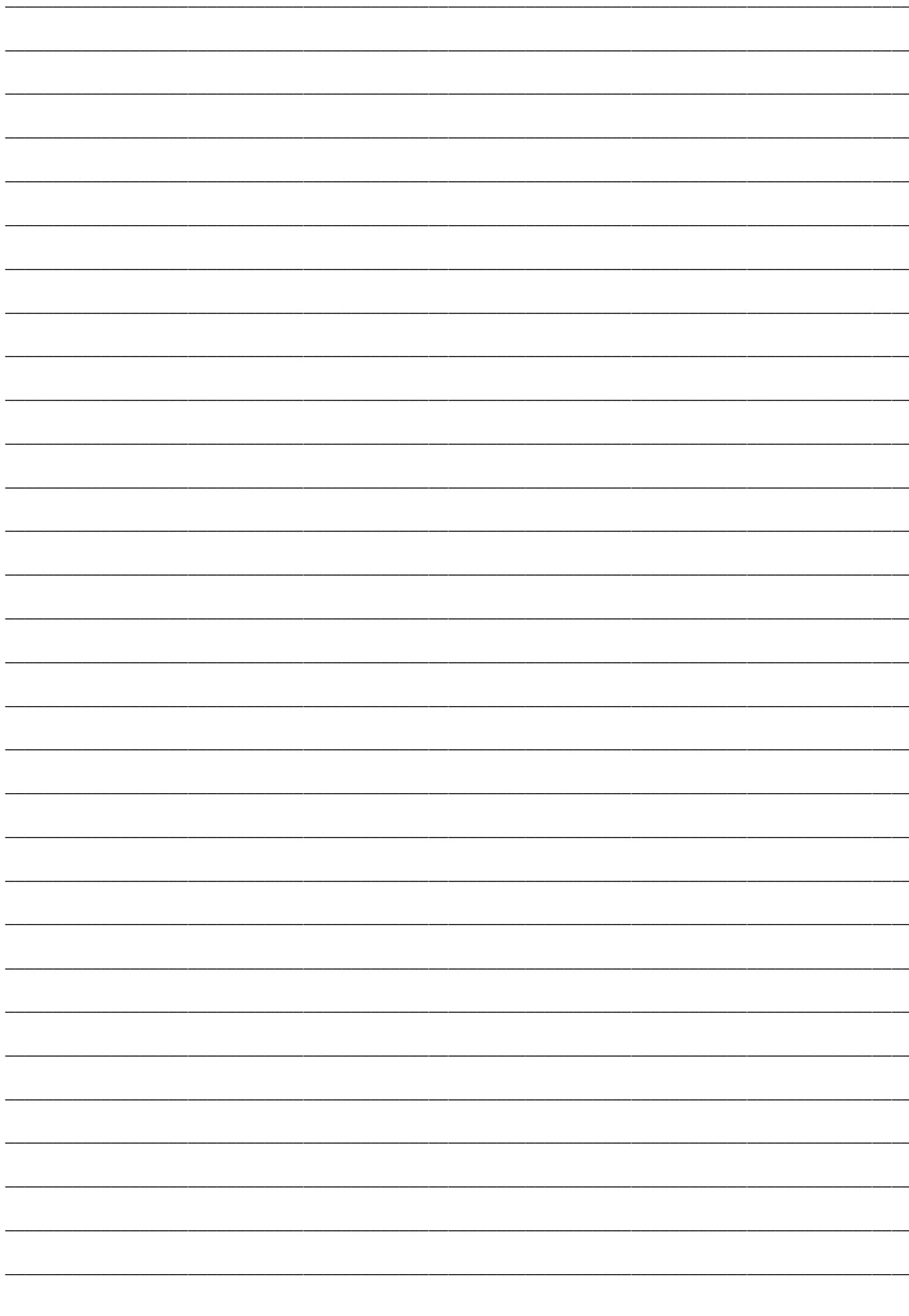
Establece la misma escala en cada eje. Debes introducirlo tras un plot o fplot.

```
>> x1=0:5
>> y1=[102.9, 75.8, 56.1, 42.2, 31.1, 23.6]
>> plot(x1, y1, 'r*')
>> syms x
>> f(x)=102.04*exp(-0.29*x), pretty(f)
>> hold on
>> fplot(f, [0, 5])
```

VECTORES FILA

$V = [a : h : b]$

- a) Primer valor
- b) Segundo valor
- h) Salto entre valor y valor



LÍMITES

<code>limit(f,x,a)</code>	Calcula el límite de f cuando x tiende hacia el punto a : $\lim_{x \rightarrow a} f(x)$. Si no se especifica la variable, se tomará, por defecto, la variable x . Si no se especifica a se tomará, por defecto, 0.
<code>limit(f,x,a,'right')</code>	Calcula el límite de f en a por la derecha: $\lim_{x \rightarrow a^+} f(x)$.
<code>limit(f,x,a,'left')</code>	Calcula el límite de f en a por la izquierda: $\lim_{x \rightarrow a^-} f(x)$.
<code>limit(f,x,Inf)</code>	Calcula el límite de f en <i>más infinito</i> : $\lim_{x \rightarrow +\infty} f(x)$.
<code>limit(f,x,-Inf)</code>	Calcula el límite de f en <i>menos infinito</i> : $\lim_{x \rightarrow -\infty} f(x)$.

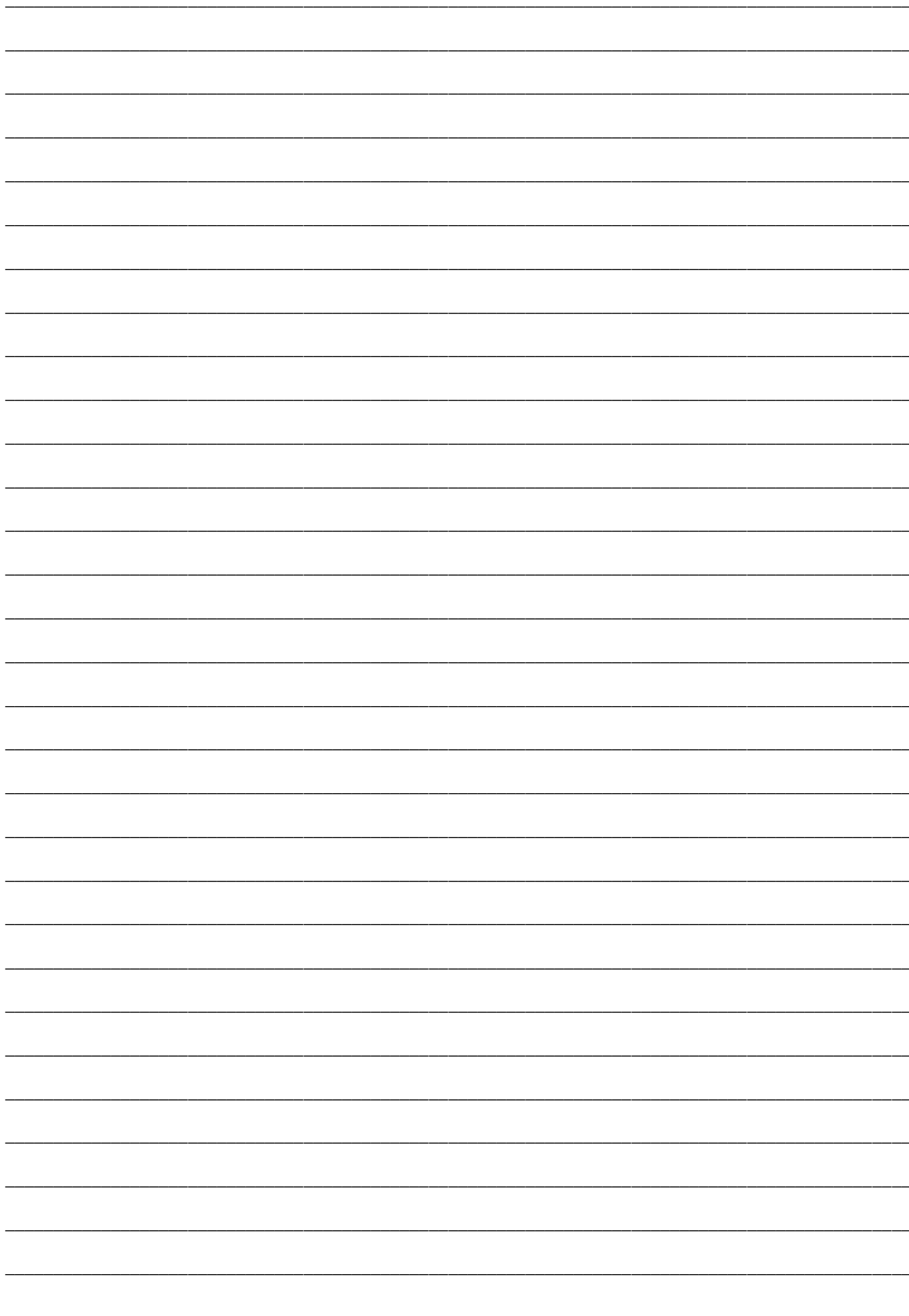
```
>> syms x
>> f(x)=sin(5*x)/log(1+4*x); pretty(f)
>> limit(f(x),x,0)
>> g(x)=log(1+sin(4*x))/(exp(sin(5*x))-1); pretty(g)
>> limit(g(x),x,0)
```

DERIVADAS

<code>diff(f,x)</code>	Calcula la derivada de la expresión simbólica f con respecto a la variable x .
<code>diff(f)</code>	Deriva f con respecto a la variable que elegirá MATLAB por defecto.
<code>diff(f,x,n)</code>	Calcula la derivada n -ésima de f con respecto a la variable x .

TAYLOR

<code>taylor(f,x,a,'Order',n)</code>	Calcula el <i>Polinomio de Taylor</i> de f de grado $n - 1$ alrededor del punto a .
<code>taylor(f,'Order',n)</code>	Calcula el <i>Polinomio de MacLaurin</i> de f de grado $n - 1$.
<code>taylortool</code>	Es una calculadora interactiva de <i>Polinomios de Taylor</i> .



INTEGRALES

<code>int(f,x)</code>	Calcula la integral indefinida $\int f(x)dx$, pero sin la constante aditiva, es decir, calcula una primitiva de f . Si no se especifica la variable, MATLAB elegirá una por defecto, teniendo siempre preferencia por la variable x .
<code>int(f,x,a,b)</code>	Calcula la integral definida $\int_a^b f(x)dx$, siendo f una expresión simbólica.
<code>int(f(x),x,a,b)</code>	Calcula lo mismo que antes siendo $f(x)$ una función simbólica.

Un ejemplo:

1. Calculamos los posibles puntos de corte entre la curva y la recta, llamando f a la expresión de la curva y g a la recta, tenemos:

```
>> syms x
>> f(x)=2*x^3
>> g(x)=8*x
>> solve(f-g)
```

2. Representamos f y la recta en el intervalo $[-2, 2]$ (dicho intervalo contiene a todas las raíces):

```
>> fplot(f,[-2,2])
>> hold on
>> fplot(g,[-2,2])
>> hold off
```

3. En el gráfico anterior podemos ver que f está por encima de la recta en $[-2, 0]$ y por debajo de la recta en $[0, 2]$. Para hallar el área necesitamos resolver la integral:

$$\int_{-2}^2 |f(x) - g(x)| dx = \int_{-2}^0 (f(x) - g(x)) dx + \int_0^2 (g(x) - f(x)) dx$$

cuyo cálculo con MATLAB sería:

```
>> a1=int(f(x)-g(x),x,-2,0)
a1 = 8
>> a2=int(g(x)-f(x),x,0,2)
a2 =8
>> area=a1+a2
area =16
```

Puesto que MATLAB posee la función valor absoluto, podríamos calcular esta área directamente:

```
>> area=int(abs(f(x)-g(x)),x,-2,2)
area =16
```

ÁREA

<code>area(x,y)</code>	Dibuja y colorea el área encerrada entre la gráfica de una función y el eje de
<code>area(x,f(x))</code>	las x , en el intervalo de representación.

Por ejemplo, dibujar y colorear el área encerrada por la función $f(x) = \sin(x)$ y el eje x en $[-\pi, \pi]$ utilizamos las órdenes:

```
>> syms x;  
>> f(x)=sin(x);  
>> xd = linspace(-pi,pi,50);
```

```
>> yd=double(f(xd));  
>> area(xd,yd) % se DIBUJA sombreada  
>> valor_area_encerrada=2*int(f,x,0,pi)  
>> % Anotaciones voluntarias...  
>> hold on  
>> title('Area calculada f(x) = sen(x) entre -pi y pi')  
>> legend('area')
```